Relatorio de Analise - Projeto Bomberman

Relatorio de Analise - Projeto Bomberman

Projeto analisado:bomber-manDiagrama analisado:docs/diagrama.puml e docs/diagrama_bomberman.svgCodigo analisado:src/**/*.javaData da analise: 12/11/2026

Resumo Executivo

O diagrama de classes descreve uma versao completa do jogo Bomberman, com modelo, interface grafica, persistencia, excecoes e regras de jogo. O codigo atual, porem, ainda esta em fase inicial: muitas classes existem apenas como esqueleto e algumas implementacoes ainda nao seguem o contrato do diagrama.

A compilacao falha imediatamente em Bomba.java por causa de uma virgula sobrando no construtor. Ao testar em uma copia temporaria com essa virgula removida, aparecem outros erros importantes: construtores de subclasses nao chamam o construtor de ElementoMapa, Bomba nao implementa getSimbolo(), e Personagem.mover declara uma excecao que a interface IMovel nao permite.

Passo 1 - Como Ler o Diagrama de Classes

1. Pacotes

O diagrama separa o projeto em blocos de responsabilidade:

Pacote | Responsabilidade

modelo | Contem as regras centrais do jogo: jogador, inimigos, bombas, tabuleiro, blocos, itens e estado da partida.

interfaces | Define comportamentos reutilizaveis, como mover, receber dano e controlar tempo.

persistencia | Cuida de carregar mapas, salvar jogo e manter ranking.

gui | Representa a interface grafica com Swing, usando janela principal, painel do jogo e painel do menu.

excecoes | Organiza erros especificos do jogo, todos baseados em JogoException.

2. Classe base: ElementoMapa

ElementoMapa e a classe abstrata que representa qualquer coisa posicionada no mapa. Ela guarda uma Posicao, permite ler e alterar essa posicao, e obriga as subclasses a informarem um simbolo por meio de getSimbolo().

No diagrama, Bloco, Bomba, Item e Personagem herdam de ElementoMapa. Isso cria uma regra simples: tudo que aparece no mapa deve saber onde esta e como se representar.

3. Posicao e Celula

Posicao guarda coordenadas de linha e coluna. Celula representa uma casa do tabuleiro e pode conter ate um bloco, uma bomba e um item. Ela tambem sabe dizer se pode ser atravessada e qual simbolo deve aparecer naquela casa.

4. Tabuleiro

Tabuleiro contem uma matriz de Celula. Seus metodos principais deveriam buscar uma celula por posicao, verificar se a posicao e valida, testar se esta livre e imprimir o mapa.

5. Blocos

Bloco e abstrata e guarda se o bloco e transponivel. BlocoIndestrutivo representa parede fixa, normalmente com simbolo #. BlocoDestrutivo representa caixa ou obstaculo que pode receber dano, ser destruido e esconder um item.

6. Interfaces

Interface | Quem usa | Sentido no jogo

IMovel | Personagem | Define que o objeto pode se mover em uma direcao no tabuleiro.

IDestrutivel | Personagem e BlocoDestrutivo | Define que o objeto pode receber dano e informar se foi destruido.

ITemporizavel | Bomba | Define contagem de tempo por tick() e consulta de tempo restante.

7. Personagens

Personagem e a base de Jogador e Inimigo. Ele possui o atributo vivo, pode se mover e pode receber dano. No diagrama, esse e o ponto comum para tudo que tem vida e ocupa o mapa.

Jogador concentra dados como nome, vidas, pontuacao, limite de bombas e raio da explosao. Ele pode plantar bomba, perder vida e receber melhorias.

Inimigo e abstrata e define a acao geral do inimigo. As subclasses mudam a inteligencia: InimigoComum decide movimentos simples, InimigoRastreador persegue o jogador, e InimigoSabotador procura bombas e pode transforma-las em armadilhas.

8. Bombas e explosoes

Bomba pertence a um Jogador, tem raio, tempo restante e tempo total. Ela deveria implementar Runnable e ITemporizavel, permitindo contagem de tempo e explosao. Ao explodir, cria uma Explosao, que guarda a origem e as celulas atingidas.

BombaArmadilha herda de Bomba e possui um Bloco usado como disfarce. No diagrama, ela e criada pelo InimigoSabotador.

9. Itens e power-ups

Item e a base para objetos coletaveis. PowerUp e um tipo de item que aplica efeito ao jogador. PowerUpBomba aumenta o limite de bombas, PowerUpRaio aumenta o raio da explosao, e PortaSaida representa a saida da fase.

10. Jogo

Jogo e a classe controladora principal. Ela agrega jogador, inimigos, bombas, tabuleiro, dificuldade, estado, turno, dicas e modo debug. Os metodos do diagrama mostram que ela deveria iniciar a partida, processar comandos, mover jogador, mover inimigos, atualizar bombas, verificar colisao e detectar vitoria ou derrota.

11. Persistencia

GeradorMapa deveria carregar ou gerar tabuleiros, espalhar caixas, itens e inimigos. GerenciadorArquivo deveria salvar e carregar jogos e rankings. Ranking guarda registros, e RegistroRanking deveria implementar Comparable para ordenar pontuacoes.

12. Interface grafica

O diagrama coloca a interface grafica no pacote gui. JanelaPrincipal herda de JFrame e contem PainelJogo e PainelMenu. PainelJogo desenha o jogo com paintComponent, e PainelMenu oferece acoes como jogar, carregar, ver ranking e sair.

13. Relacionamentos importantes

Relacionamento | Significado

Heranca: seta com triangulo | Uma classe especializada reaproveita a classe base. Exemplo: Jogador herda de Personagem.

Implementacao de interface | A classe promete cumprir os metodos da interface. Exemplo: Bomba deveria implementar ITemporizavel.

Composicao: losango preenchido | Um objeto faz parte do outro e depende dele. Exemplo: Jogo contem Tabuleiro.

Agregacao: losango vazio | Um objeto referencia outro, mas nao necessariamente controla seu ciclo de vida. Exemplo: Celula pode conter Bomba.

Dependencia: seta pontilhada | Uma classe usa outra em algum metodo. Exemplo: GeradorMapa e usado por Jogo.

Passo 2 - Erros Encontrados no Codigo

1. Erro que impede a compilacao imediatamente

Arquivo | Linha | Problema | Como corrigir

src/modelo/Bomba.java | 10 | Erro sintatico:Bomba(Jogador dono,) tem uma virgula sobrando. | Trocar para algo como Bomba(Jogador dono) ou completar os parametros esperados.

src/modelo/Bomba.java:10: error: illegal start of type
Bomba(Jogador dono,) {
^
1 error

2. Erros que aparecem apos remover temporariamente a virgula

Estes erros foram verificados em uma copia temporaria, sem alterar o projeto original.

Arquivo | Problema | Motivo | Correcao sugerida

Bloco.java | Construtor nao chama super(posicao). | ElementoMapa exige uma Posicao no construtor. | Alterar o construtor de Bloco para receber Posicao e chamar super(posicao).

Bomba.java | Bomba nao implementa getSimbolo(). | ElementoMapa tem metodo abstrato getSimbolo(). | Implementar public char getSimbolo(), por exemplo retornando 'B'.

Bomba.java | Construtor nao chama super(posicao). | Como Bomba herda de ElementoMapa, precisa inicializar a posicao. | Adicionar Posicao posicao ao construtor e chamar

super(posicao).

Personagem.java | mover declara throws MovimentoInvalidoException, mas a interface IMovel nao declara. | Uma implementacao nao pode adicionar excecao checada que nao esta no metodo da interface. | Adicionar throws MovimentoInvalidoException em IMovel.mover ou remover esse throws da implementacao.

/tmp/bomberman-compile-check/src/modelo/Bloco.java:6: error: constructor ElementoMapa in class ElementoMapa cannot be applied to given types;
/tmp/bomberman-compile-check/src/modelo/Personagem.java:21: error: mover(Direcao,Tabuleiro) in Personagem cannot implement mover(Direcao,Tabuleiro) in IMovel
/tmp/bomberman-compile-check/src/modelo/Bomba.java:3: error: Bomba is not abstract and does not override abstract method getSimbolo() in ElementoMapa
/tmp/bomberman-compile-check/src/modelo/Bomba.java:10: error: constructor ElementoMapa in class ElementoMapa cannot be applied to given types;
/tmp/bomberman-compile-check/src/modelo/Inimigo.java:3: error: mover(Direcao,Tabuleiro) in Personagem cannot implement mover(Direcao,Tabuleiro) in IMovel

3. Problemas de logica ou implementacao incompleta

Arquivo | Observacao | Impacto

ElementoMapa.java | setPosicao(Posicao p) faz this.posicao = posicao em vez de usar p. | A posicao nunca sera alterada corretamente.

Bloco.java | isTransponivel() sempre retorna false, ignorando o atributo. | Blocos marcados como transponiveis continuarao bloqueando passagem.

BlocoDestrutivo.java | Existe itemOculto, mas nao ha atributo destruido nem receberDano. | O bloco nao cumpre o comportamento esperado pelo diagrama.

Personagem.java | mover e receberDano estao vazios. | Personagens nao se movem nem sofrem dano de verdade.

Jogo.java | Classe esta vazia. | Falta o controlador principal do jogo.

Tabuleiro.java | Classe esta vazia. | Falta a matriz do mapa e as regras de ocupacao.

Jogador.java | Classe esta vazia. | Faltam vidas, pontuacao, bomba, raio e acoes do jogador.

Item.java | Esta como classe concreta vazia, mas o diagrama espera classe abstrata. | Power-ups e porta de saida ficam sem contrato comum de coleta.

PowerUp.java | Esta abstrata, mas nao define aplicar nem coletar. | Power-ups nao tem efeito no jogador.

InimigoComum.java, InimigoSabotador.java, InimigoRastejador.java | Nao herdam de Inimigo. | As subclasses de inimigo nao funcionam polimorficamente.

BombaArmadilha.java | Nao herda de Bomba. | A armadilha nao reaproveita o comportamento de bomba.

RegistroRanking.java | Nao implementa Comparable. | Ranking nao pode ser ordenado como previsto.

Main.java | Apenas imprime Funciona. | A aplicacao ainda nao inicia menu, janela ou jogo.

4. Divergencias entre diagrama e codigo

No diagrama | No codigo | Consequencia

Pacote gui | Classes de interface estao em modelo. | A arquitetura real nao bate com o diagrama.

PainelJogo | Arquivo/classe PaineldoJogo. | Nome diferente dificulta rastrear relacoes e pode gerar erro quando a GUI for implementada.

InimigoRastreador | InimigoRastejador. | O nome nao corresponde ao diagrama nem ao comportamento descrito de perseguidor.

Bomba implements Runnable, ITemporizavel | Bomba apenas herda de ElementoMapa. | Faltam tempo, tick, run e contrato temporizado.

BlocoDestrutivo implements IDestrutivel | Nao implementa a interface. | O codigo nao garante os metodos de destruicao.

Classes Swing herdam de JFrame/JPanel | Classes GUI sao vazias e nao herdam de Swing. | A interface grafica ainda nao existe funcionalmente.

Passo 3 - Informacoes Uteis para Corrigir

Ordem recomendada de correcao

- Corrigir o erro sintatico de Bomba.java.
- Decidir o construtor padrao de ElementoMapa: ou todas as subclasses recebem Posicao, ou ElementoMapa ganha um construtor vazio protegido.
- Alinhar interfaces e excecoes, principalmente IMovel.mover e MovimentoInvalidoException.
- Implementar os metodos abstratos obrigatorios, especialmente getSimbolo().
- Fazer as herancas baterem com o diagrama: inimigos, bomba armadilha, itens, power-ups e ranking.
- Implementar Tabuleiro e Celula, porque o resto do jogo depende disso.
- Implementar Jogo como controlador central.
- Depois disso, integrar Main, GUI e persistencia.

Comandos usados na verificacao

javac -d /tmp/bomberman-build $(find src -name '*.java')

Resultado: falha em Bomba.java por erro sintatico.

javac -d /tmp/bomberman-build2 $(find /tmp/bomberman-compile-check/src -name '*.java')

Resultado: apos remover a virgula em copia temporaria, surgem cinco erros adicionais de contrato/heranca.

Checklist para comparar codigo e diagrama

- Cada classe do diagrama existe no codigo?
- O nome da classe esta igual ao diagrama?
- O pacote esta igual ao diagrama?
- As herancas estao implementadas com extends?
- As interfaces estao implementadas com implements?
- Todo metodo abstrato foi implementado nas classes concretas?

- Os construtores chamam corretamente super(...)?
- As excecoes checadas aparecem tambem nas interfaces, quando necessario?

Conclusao

O diagrama esta bem mais completo que o codigo atual e serve como um bom roteiro de implementacao. Neste momento, o projeto ainda nao esta pronto para execucao real do jogo: primeiro e necessario corrigir a compilacao, alinhar herancas/interfaces e implementar as classes centrais Tabuleiro, Jogo, Jogador, Bomba e Personagem.